Challenge-VPC_Transit-Gateway_Endpoints

Use Case: Setting up Transit Gateway and VPC Endpoints for a Multi-VPC Architecture

Scenario:

A large organization is migrating its on-premises infrastructure to the AWS cloud.

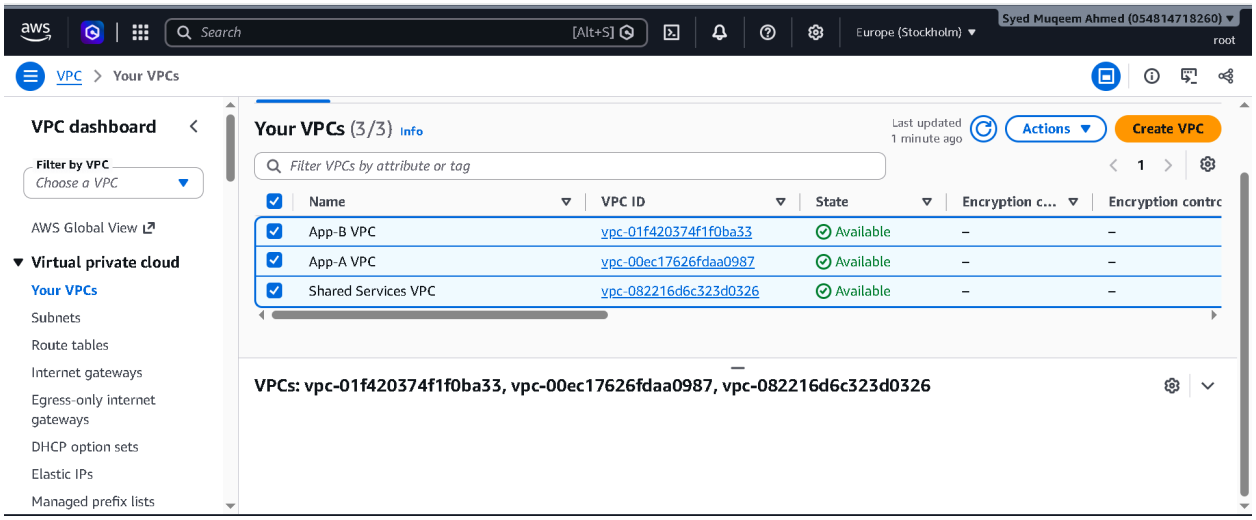
The organization's architecture involves multiple VPCs for different departments and applications, each requiring secure communication with centralized services and external resources.

The IT team needs to design and implement a scalable and efficient network architecture to accommodate the organization's growth and ensure robust connectivity between VPCs and external services.

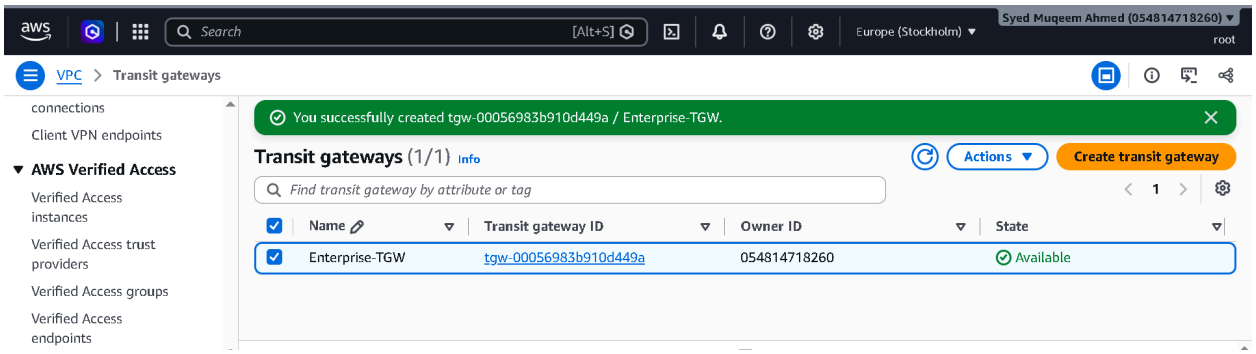
Objectives:

- Design and deploy a scalable network architecture using AWS Transit Gateway to simplify network connectivity between multiple VPCs.

Create VPCs



create_Transit-gateway



Step 3: Create Transit Gateway Attachments

Connect each VPC to your Transit Gateway:

Go to **Transit Gateway Attachments** → **Create Transit Gateway Attachment**.

Select your Enterprise-TGW.

Choose **VPC** as the attachment type.

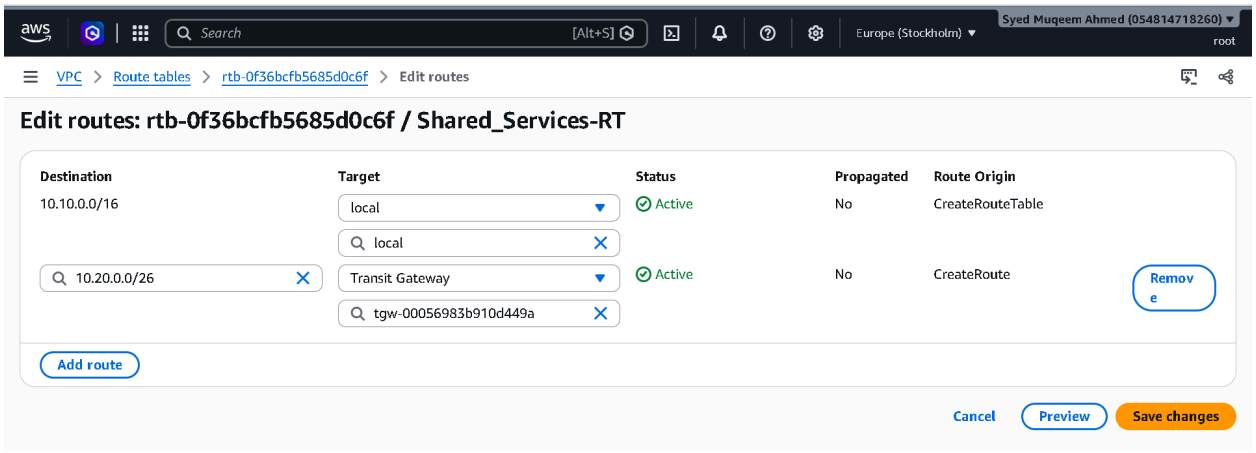
Select your VPC (App-A VPC) and choose the **Private Subnets** across your Availability Zones for high availability.

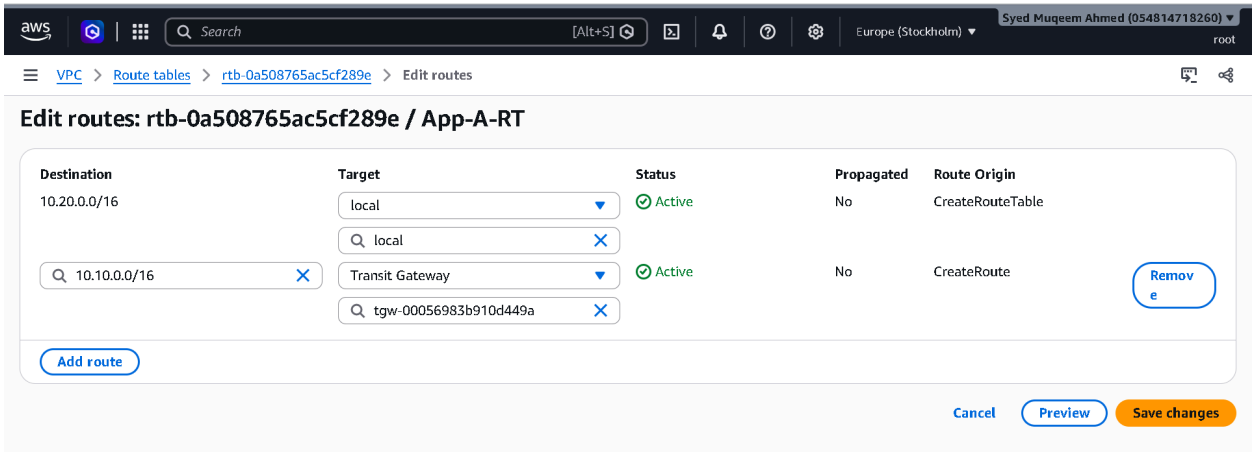
Repeat this process for App-B VPC and Shared Services VPC.

Step 4: Update VPC Route Tables for Inter-VPC Traffic

To allow `App-A VPC` to communicate with `Shared Services VPC` via the TGW:

1. Go to your **VPC Route Tables** associated with the private subnets of `App-A VPC`.
2. Add a new route:
    - **Destination:** `10.10.0.0/16` (CIDR of Shared Services VPC)
    - **Target:** Select your Transit Gateway (`tgw-xxxxxx`)
3. Mirror this configuration in the Shared Services VPC route table pointing back to `App-A` (`10.20.0.0/16`).





ec2-user@ip-10-10-1-37:~

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/Downloads
$ ssh -i cloud-testing.pem ec2-user@16.16.185.142
The authenticity of host '16.16.185.142 (16.16.185.142)' can't be established.
ED25519 key fingerprint is: SHA256:MYj4in5OCdvRPtljvZtdQrmDjWYEETP2eqY3cnnSAMg
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '16.16.185.142' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-1-37 ~]$ ping 10.20.2.144
PING 10.20.2.144 (10.20.2.144) 56(84) bytes of data.
^C
--- 10.20.2.144 ping statistics ---
693 packets transmitted, 0 received, 100% packet loss, time 719675ms

[ec2-user@ip-10-10-1-37 ~]$ ping 10.20.2.144
PING 10.20.2.144 (10.20.2.144) 56(84) bytes of data.
^C
--- 10.20.2.144 ping statistics ---
110 packets transmitted, 0 received, 100% packet loss, time 113341ms

[ec2-user@ip-10-10-1-37 ~]$ ping 10.20.2.144
PING 10.20.2.144 (10.20.2.144) 56(84) bytes of data.
^C
--- 10.20.2.144 ping statistics ---
24 packets transmitted, 0 received, 100% packet loss, time 23942ms

[ec2-user@ip-10-10-1-37 ~]$ ping 10.30.1.160
PING 10.30.1.160 (10.30.1.160) 56(84) bytes of data.
64 bytes from 10.30.1.160: icmp_seq=1 ttl=126 time=2.02 ms
64 bytes from 10.30.1.160: icmp_seq=2 ttl=126 time=0.400 ms
64 bytes from 10.30.1.160: icmp_seq=3 ttl=126 time=0.447 ms
64 bytes from 10.30.1.160: icmp_seq=4 ttl=126 time=0.357 ms
64 bytes from 10.30.1.160: icmp_seq=5 ttl=126 time=0.359 ms
64 bytes from 10.30.1.160: icmp_seq=6 ttl=126 time=0.388 ms
64 bytes from 10.30.1.160: icmp_seq=7 ttl=126 time=0.382 ms
64 bytes from 10.30.1.160: icmp_seq=8 ttl=126 time=0.359 ms
```

```
[ec2-user@ip-10-10-1-37 ~]$ ping 10.20.1.28
PING 10.20.1.28 (10.20.1.28) 56(84) bytes of data.
64 bytes from 10.20.1.28: icmp_seq=1 ttl=126 time=0.927 ms
64 bytes from 10.20.1.28: icmp_seq=2 ttl=126 time=0.393 ms
64 bytes from 10.20.1.28: icmp_seq=3 ttl=126 time=0.384 ms
64 bytes from 10.20.1.28: icmp_seq=4 ttl=126 time=0.370 ms
64 bytes from 10.20.1.28: icmp_seq=5 ttl=126 time=0.323 ms
64 bytes from 10.20.1.28: icmp_seq=6 ttl=126 time=0.330 ms
64 bytes from 10.20.1.28: icmp_seq=7 ttl=126 time=0.349 ms
64 bytes from 10.20.1.28: icmp_seq=8 ttl=126 time=0.382 ms
64 bytes from 10.20.1.28: icmp_seq=9 ttl=126 time=0.338 ms
64 bytes from 10.20.1.28: icmp_seq=10 ttl=126 time=0.342 ms
64 bytes from 10.20.1.28: icmp_seq=11 ttl=126 time=0.351 ms
^C
--- 10.20.1.28 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10339ms
rtt min/avg/max/mdev = 0.323/0.408/0.927/0.165 ms
[ec2-user@ip-10-10-1-37 ~]$
```

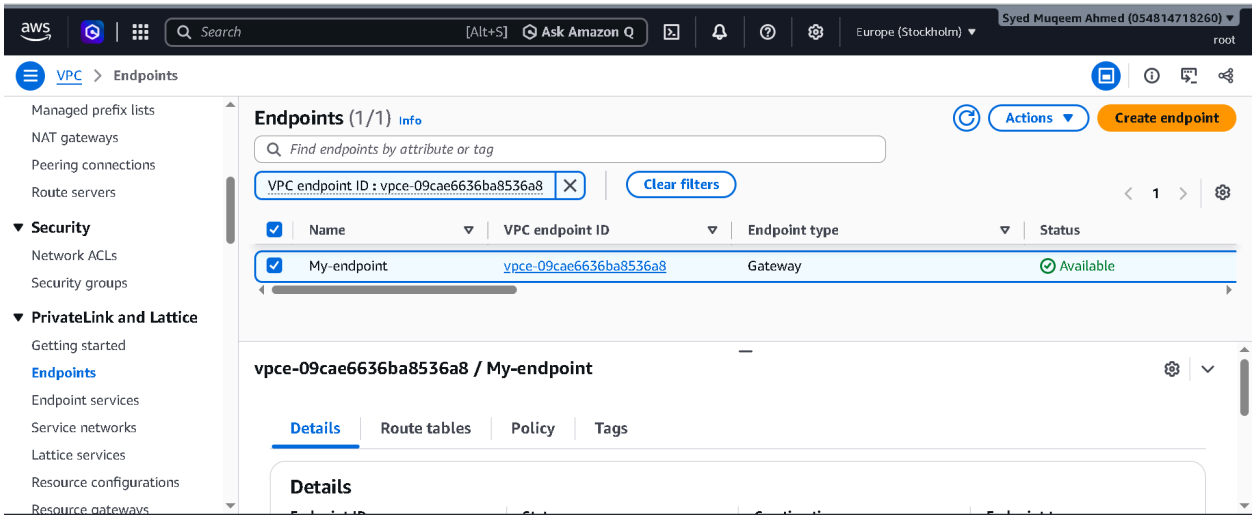
- Configure VPC endpoints to securely access AWS services without internet gateways or NAT gateways, ensuring data privacy and minimizing exposure to external threats.

Step 5: Configure VPC Endpoints for Private Access

To allow private subnets to securely access AWS services (like Amazon S3 or Systems Manager):

For Amazon S3 (Gateway Endpoint):

1. Go to **VPC Console** → **Endpoints** → **Create Endpoints**.
2. Select **AWS services** and search for com.amazonaws.eu-north-1.s3.
3. Choose the **Gateway** type (free, uses route tables).
4. Select your VPC and check the boxes for the private route tables.



ec2-user@ip-10-10-1-37:~

```
[ec2-user@ip-10-10-1-37 ~]$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: AKIAQZQZWHE2EH7QKT7W
AWS Secret Access Key [None]: DkewaROWsJx/MaOsaLBMc//2zZe/7OikDnz2S69U
Default region name [None]:
Default output format [None]: json
[ec2-user@ip-10-10-1-37 ~]$ aws s3 ls
2026-11-03 06:55:32 muq-s3-bucket
[ec2-user@ip-10-10-1-37 ~]$
```